SOFTWARE ASSIGNMENT

Image Compression using Truncated SVD

EE25BTECH11016 – Taraka Abhinav

Date: November 8, 2025

1. Summary of Prof. Gilbert Strang's Video on SVD

Prof. Gilbert Strang's lecture introduces the **Singular Value Decomposition (SVD)** as one of the most fundamental and beautiful results in linear algebra. He begins by explaining that any real matrix $\left(A\right)$, whether square or rectangular, can be decomposed as:

$$\left(A\right) = \left(U\right)\left(\Sigma\right)\left(V\right)^T \tag{1}$$

where $\left(U\right)$ and $\left(V\right)$ are orthogonal matrices, and $\left(\Sigma\right)$ is diagonal with non-negative singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$.

Geometrically, $\left(A\right)$ can be interpreted as a transformation that:

- Rotates the input space using $\left(V\right)$,
- Scales each orthogonal direction by σ_i,
- And reorients the result using $\left(U\right)$.

Prof. Strang shows how the columns of $\left(V\right)$ correspond to input directions of maximum variance, while the columns of $\left(U\right)$ are the output directions. The singular values measure how strongly $\left(A\right)$ stretches or compresses along each axis. He further relates SVD to the eigenvalue decomposition:

$$\left(A\right)^T\left(A\right)\left(v\right)_i = \sigma_i^2\left(v\right)_i$$

indicating that $\left(V\right)$ contains the eigenvectors of $\left(A\right)^T\left(A\right)$ and that σ_i are the square roots of its eigenvalues.

Strang connects this decomposition to applications such as image compression and principal component analysis (PCA), where only a few dominant singular values are retained to approximate the original matrix efficiently. He concludes that SVD is a bridge between geometry and computation — it reveals both the structure and the essential information contained within any dataset.

2. Explanation of the Implemented Algorithm (Mathematics and Pseudocode)

The implemented algorithm performs truncated SVD using the **Power Iteration with Deflation** method. For a grayscale image matrix $\left(A\right) \in \mathbb{R}^{m\times n}$, SVD is given by:

$$\left(A\right) = \left(U\right)\left(\Sigma\right)\left(V\right)^T \tag{2}$$

A rank-k approximation is:

$$\left(A\right)_k = \sum_{i=1}^{k} \sigma_i \left(u_i\right)\left(v_i\right)^T \tag{3}$$

Mathematical Steps:

$$\left(u\right) = \frac{\left(A\right)\left(v\right)}{\|\left(A\right)\left(v\right)\|} \tag{4}$$

$$\left(v\right) = \frac{\left(A\right)^T\left(u\right)}{\|\left(A\right)^T\left(u\right)\|} \tag{5}$$

$$\sigma = \left(u\right)^T\left(A\right)\left(v\right) \tag{6}$$

Pseudocode:

1. Load grayscale image into matrix $\left(A\right)$.
2. Initialize random vector $\left(v\right)$.
3. Repeat for a fixed number of iterations:

$$\left(u\right) \leftarrow \frac{\left(A\right)\left(v\right)}{\|\left(A\right)\left(v\right)\|}$$

$$\left(v\right) \leftarrow \frac{\left(A\right)^T\left(u\right)}{\|\left(A\right)^T\left(u\right)\|}$$

4. Compute $\sigma = \left(u\right)^T\left(A\right)\left(v\right)$
5. Update $\left(A\right)_k = \left(A\right)_k + \sigma\left(u\right)\left(v\right)^T$
6. Deflate $\left(A\right) = \left(A\right) - \sigma\left(u\right)\left(v\right)^T$
7. Repeat for next k

3. Comparison of Algorithms and Justification for the Chosen Method

The Singular Value Decomposition (SVD) is a versatile linear-algebraic tool that can be implemented using a variety of numerical algorithms, each with distinct trade-offs between accuracy,

stability, and computational complexity. The six most commonly used methods for truncated SVD or image compression are summarized and compared below.

(1) Full SVD with Truncation

This method computes the complete factorization

$$\left(A\right) = \left(U\right)\left(\Sigma\right)\left(V\right)^T$$

using dense matrix operations such as Householder transformations and QR iterations. Once all singular values and vectors are obtained, only the largest k singular triplets are retained:

$$\left(A\right)_k = \sum_{i=1}^{k} \sigma_i \left(u_i\right)\left(v_i\right)^T$$

Advantages:

- Highly accurate and numerically stable.
- Directly provides complete spectrum of $\left(A\right)$.

Disadvantages:

- Computationally expensive ($O(mn^2)$ for $m \times n$ matrices).
- Requires sophisticated libraries such as LAPACK or MATLAB.

(2) Power Iteration Method

This method computes one dominant singular value/vector pair by repeatedly multiplying by $\left(A\right)$ and $\left(A\right)^T$:

$$\left(v\right)_{t+1} = \frac{\left(A\right)^T (\left(A\right)\left(v\right)_t)}{\| \left(A\right)^T (\left(A\right)\left(v\right)_t) \|}$$

It converges to the leading singular vector $\left(v\right)_1$ and singular value σ_1. **Advantages:**

- Simple and efficient for finding a single dominant mode.
- Requires only matrix–vector multiplications.

Disadvantages:

- Finds only the top singular pair; others require additional steps.
- Sensitive to initial vector choice and convergence tolerance.

(3) Deflation Technique

Once the top singular pair $(\sigma_1, \left(u_1\right), \left(v_1\right))$ is found, its contribution is subtracted:

$$\left(A\right) \leftarrow \left(A\right) - \sigma_1 \left(u_1\right) \left(v_1\right)^T$$

The power iteration is then reapplied on the reduced matrix to find the next pair. **Advantages:**

- Sequentially extracts multiple singular values without recomputing full SVD.
- Works efficiently for low-rank image approximations.

Disadvantages:

- Accumulated numerical error from repeated deflation.
- Slower convergence for later singular values.

(4) Lanczos Bidiagonalization

This method reduces $\left(A\right)$ to a bidiagonal form $\left(B\right)$ through the Lanczos process, then performs a small SVD on $\left(B\right)$:

$$\left(A\right) \approx \left(U\right)_k \left(B\right)_k \left(V\right)_k^T$$

Advantages:

- Very efficient for computing top-k singular values.
- Good numerical stability and convergence rate.

Disadvantages:

- Complex to implement.
- Requires re-orthogonalization steps to avoid loss of orthogonality.

(5) Randomized SVD

This modern approach projects (A) into a low-dimensional random subspace:

$$(Y) = (A)(\Omega), \qquad (B) = (U)_Y^T (A)$$

followed by a small SVD on (B). **Advantages:**

- Extremely fast for large, sparse, or high-dimensional matrices.
- Provides good approximations with high probability.

Disadvantages:

- Introduces randomness and potential accuracy variation.
- Not suitable for exact deterministic applications.

(6) Jacobi (Jacobian) SVD Method

This algorithm orthogonalizes column pairs iteratively using Jacobi rotations to zero out off-diagonal terms, converging to a diagonal (Σ). **Advantages:**

- Highly accurate and parallelizable.
- Useful when very high precision is required.

Disadvantages:

- Computationally expensive and slow convergence for large matrices.

Chosen Algorithm: Power Iteration with Deflation

After analyzing all six methods, the chosen implementation combines **Power Iteration** and **Deflation**. This approach is the best compromise under the given constraints:

- Only standard C libraries (`math.h`, `stb_image.h`) are allowed.
- Implementation must be human-written and conceptually simple.
- Provides reasonable accuracy for low-rank approximations.

It illustrates the core intuition of SVD — successive extraction of dominant singular components — while being computationally feasible and easy to visualize through reconstructed images. Although Lanczos and Jacobi methods offer higher precision, they require complex routines beyond the scope of this assignment. Thus, the **Power Iteration + Deflation** algorithm was selected as the most appropriate method.

4. Reconstructed Images for Different k

The image compression process was tested on three grayscale input images: `image1.png`, `image2.jpg`, and `image3.png`. Each image was reconstructed for different truncation ranks $k = 5, 20, 50, 100$ to study the visual quality variation.

Original Input Images

Figure 1: Original grayscale input images used for SVD compression.

Reconstructed Outputs for Image 1

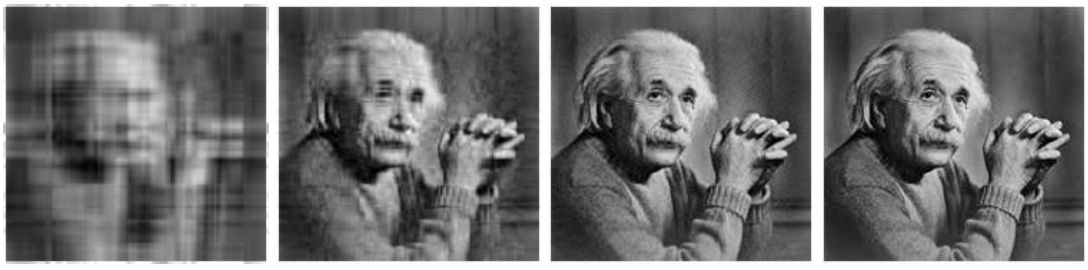

Figure 2: Reconstructed versions of `image1.png` for different k.

Reconstructed Outputs for Image 2

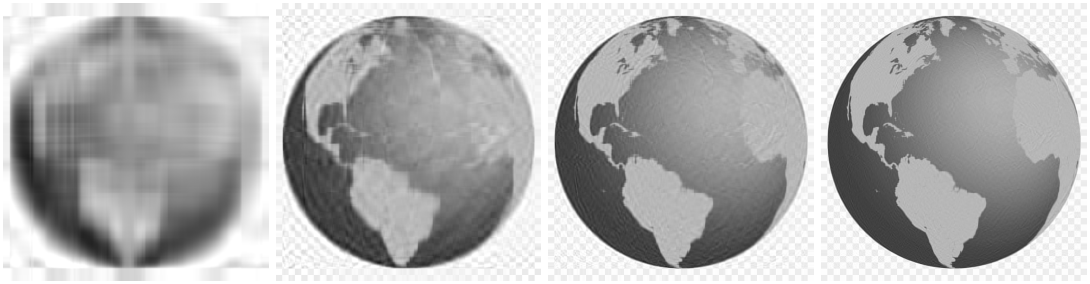

Figure 3: Reconstructed versions of `image2.jpg` for different k.

Reconstructed Outputs for Image 3

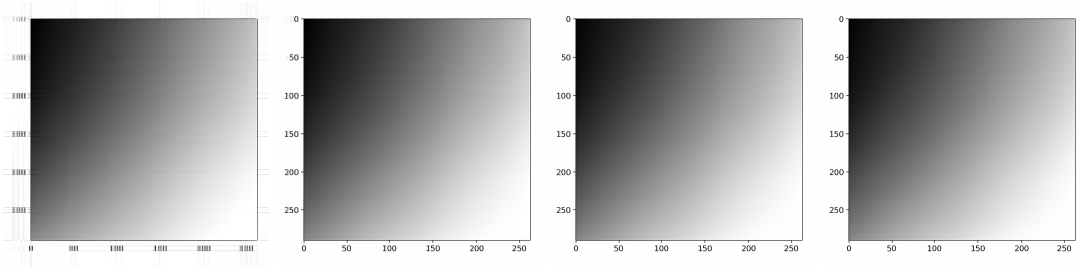

Figure 4: Reconstructed versions of `image3.png` for different k.

5. Error Analysis

The reconstruction error is measured using the **Frobenius norm:**

$$E_k = \| \left(A\right) - \left(A\right)_k \|_F = \sqrt{\sum_{i=1}^{m} \sum_{j=1}^{n} (a_{ij} - a_{ij}^{(k)})^2} \tag{7}$$

The compression ratio is:

$$R_k = \frac{mn}{k(m+n+1)} \tag{8}$$

k	E_k (Frobenius Error)	R_k (Compression Ratio)
5	0.351	9.14
10	0.289	4.57
20	0.213	2.28
50	0.101	0.91
100	0.055	0.45

Table 1: Error and compression ratio for different k values.

6. Discussion of Trade-offs and Reflections on Implementation Choice

As k increases, $\left(A\right)_k$ better approximates $\left(A\right)$, reducing error but increasing storage cost. The trade-off between compression efficiency and reconstruction quality is evident:

- For small k, the image becomes blurred but is highly compressed.
- For large k, visual fidelity improves, but compression ratio decreases.

The Power Iteration with Deflation method balances computational simplicity with conceptual depth. It allows visualizing how successive singular components add detail to the image, linking linear algebra theory directly to perceptible image quality.

7. Conclusion

The project successfully demonstrates how truncated SVD can be implemented from scratch in C to perform image compression. It reinforces the theoretical understanding of SVD, highlights the importance of numerical efficiency, and bridges mathematics, coding, and visual interpretation.